

## A Technical Introduction to Adagrad optimizer

### Section 1

**History** In 1951, Herbert Robbins and Sutton Monro introduced the earliest stochastic approximation methods, preceding stochastic gradient descent. Building on this work one year later, Jack Kiefer and Jacob Wolfowitz published an optimization algorithm very close to stochastic gradient descent, using central differences as an approximation of the gradient. Later in the 1950s, Frank Rosenblatt used SGD to optimize his perceptron model, demonstrating the first applicability of stochastic gradient descent to neural networks. Backpropagation was first described in 1986, with stochastic gradient descent being used to efficiently optimize parameters across neural networks with multiple hidden layers. Soon after, another improvement was developed: mini-batch gradient descent, where small batches of data are substituted for single samples. In 1997, the practical performance benefits from vectorization achievable with such small batches were first explored, paving the way for efficient optimization in machine learning. As of 2023, this mini-batch approach remains the norm for training neural networks, balancing the benefits of stochastic gradient descent with gradient descent. By the 1980s, momentum had already been introduced, and was added to SGD optimization techniques in 1986. However, these optimization techniques assumed constant hyperparameters, i.e. a fixed learning rate and momentum parameter. In the 2010s, adaptive approaches to applying SGD with a per-parameter learning rate were introduced with AdaGrad (for "Adaptive Gradient") in 2011 and RMSprop (for "Root Mean Square Propagation") in 2012. In 2014, Adam (for "Adaptive Moment Estimation") was published, applying the adaptive approaches of RMSprop to momentum; many improvements and branches of Adam were then developed such as Adadelta, Adagrad, AdamW, and Adamax. Within machine learning, approaches to optimization in 2023 are dominated by Adam-derived optimizers, TensorFlow and PyTorch, by far the most popular machine learning libraries, as of 2023 largely only include Adam-derived optimizers, as well as predecessors to Adam such as RMSprop and classic SGD. PyTorch also partially supports limited-memory BFGS, a line-search method, but only for single-device setups without parameter groups.

### Section 2

Each  $\{G(i,i)\}$  gives rise to a scaling factor for the learning rate that applies to a single parameter  $w_i$ . Since the denominator in this factor,  $G_i = \sum_{\tau=1}^t g_{\tau}^2$  is the  $\ell_2$  norm of previous derivatives, extreme parameter updates get dampened, while parameters that get few or

small updates receive higher learning rates. While designed for convex problems, AdaGrad has been successfully applied to non-convex optimization.

### Section 3

**Notable applications** Stochastic gradient descent is a popular algorithm for training a wide range of models in machine learning, including (linear) support vector machines, logistic regression (see, e.g., Vowpal Wabbit) and graphical models. When combined with the backpropagation algorithm, it is the de facto standard algorithm for training artificial neural networks. Its use has been also reported in the Geophysics community, specifically to applications of Full Waveform Inversion (FWI). Stochastic gradient descent competes with the L-BFGS algorithm, which is also widely used. Stochastic gradient descent has been used since at least 1960 for training linear regression models, originally under the name ADALINE. Another stochastic gradient descent algorithm is the least mean squares (LMS) adaptive filter.

### Section 4

$$v(w, t) := \gamma v(w, t-1) + (1-\gamma) \left( \nabla Q_i(w) \right)^2$$

$$\{ \displaystyle v(w,t) := \gamma v(w,t-1) + (1-\gamma) \left( \nabla Q_{i(w)} \right)^2 \}$$

### Section 5

$$\max_{k=0, \dots, \lfloor T/\eta \rfloor} \mathbb{E} [ |g(w_k) - g(W_{k\eta})| ] \leq C \eta$$

$$\{ \displaystyle \max_{k=0, \dots, \lfloor T/\eta \rfloor} \mathbb{E} [ |g(w_k) - g(W_{k\eta})| ] \leq C \eta \}$$

### Section 6

subject to additional stochastic noise. This approximation is only valid on a finite time-horizon in the following sense: assume that all the coefficients  $Q_i$  are sufficiently smooth. Let  $T > 0$  and  $g: \mathbb{R}^d \rightarrow \mathbb{R}$  be a sufficiently smooth test function. Then, there exists a constant  $C > 0$  such that for all  $\eta > 0$

### Section 7

where,  $\gamma$  is the forgetting factor. The concept of storing the historical gradient as sum of squares is borrowed from Adagrad, but "forgetting" is introduced to solve Adagrad's diminishing learning rates in non-convex problems by gradually decreasing the influence of old data. And the parameters are updated as,

### Section 8

$$Q(w) = \sum_{i=1}^n Q_i(w) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (w_1 + w_2 x_i - y_i)^2$$

$$\{ \displaystyle Q(w) = \sum_{i=1}^n Q_i(w) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (w_1 + w_2 x_i - y_i)^2 \}$$

## Section 9

for  $\Sigma(w) = \frac{1}{n^2} \left( \sum_{i=1}^n Q_i(w) - Q(w) \right) \left( \sum_{i=1}^n Q_i(w) - Q(w) \right)^T$   $\{\displaystyle \Sigma(w) = \frac{1}{n^2} \left( \sum_{i=1}^n Q_i(w) - Q(w) \right) \left( \sum_{i=1}^n Q_i(w) - Q(w) \right)^T\}$  where  $dB_t$  denotes the Ito-integral with respect to a Brownian motion is a more precise approximation in the sense that there exists a constant  $C > 0$  such that

## Section 10

Backtracking line search Backtracking line search is another variant of gradient descent. All of the below are sourced from the mentioned link. It is based on a condition known as the Armijo–Goldstein condition. Both methods allow learning rates to change at each iteration; however, the manner of the change is different. Backtracking line search uses function evaluations to check Armijo's condition, and in principle the loop in the algorithm for determining the learning rates can be long and unknown in advance. Adaptive SGD does not need a loop in determining learning rates. On the other hand, adaptive SGD does not guarantee the "descent property" – which Backtracking line search enjoys – which is that  $f(x_{n+1}) \leq f(x_n)$  for all  $n$ . If the gradient of the cost function is globally Lipschitz continuous, with Lipschitz constant  $L$ , and learning rate is chosen of the order  $1/L$ , then the standard version of SGD is a special case of backtracking line search.